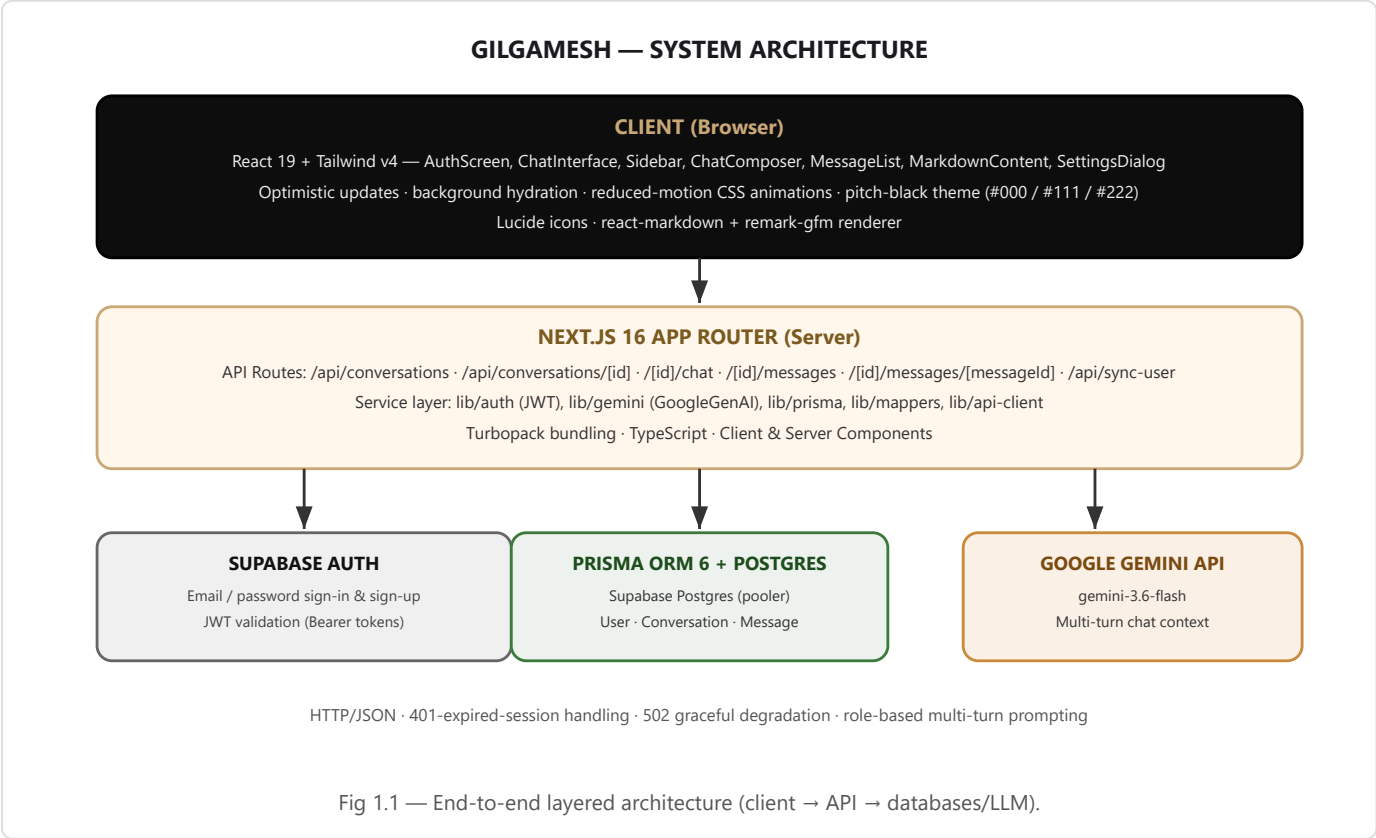


Gilgamesh — An AI Chat Workspace

Structural Diagrams — System, Data, and Process Architecture

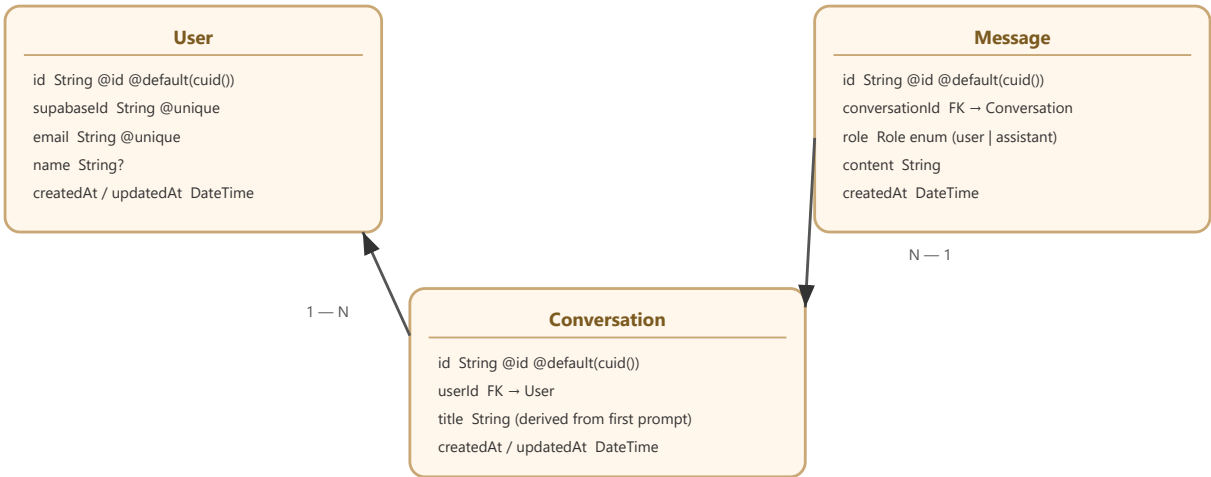
1. System Architecture Overview

The application follows a layered client–server architecture built on the Next.js App Router. The React client talks exclusively to the application's own API routes; API routes mediate between Supabase Authentication, the Prisma-managed PostgreSQL database (hosted on Supabase), and the Google Gemini API.



2. Database Entity-Relationship Diagram (Prisma Schema)

ENTITY-RELATIONSHIP DIAGRAM — PostgreSQL (Supabase)



Indexes: Conversation[userId], Conversation[updatedAt], Message[conversationId], Message[conversationId, createdAt] · Cascade deletes on both relations

Fig 2.1 — Relational model: one User owns many Conversations; each Conversation owns many Messages.

3. Authentication Flow

AUTHENTICATION FLOW (SUPABASE EMAIL / PASSWORD)

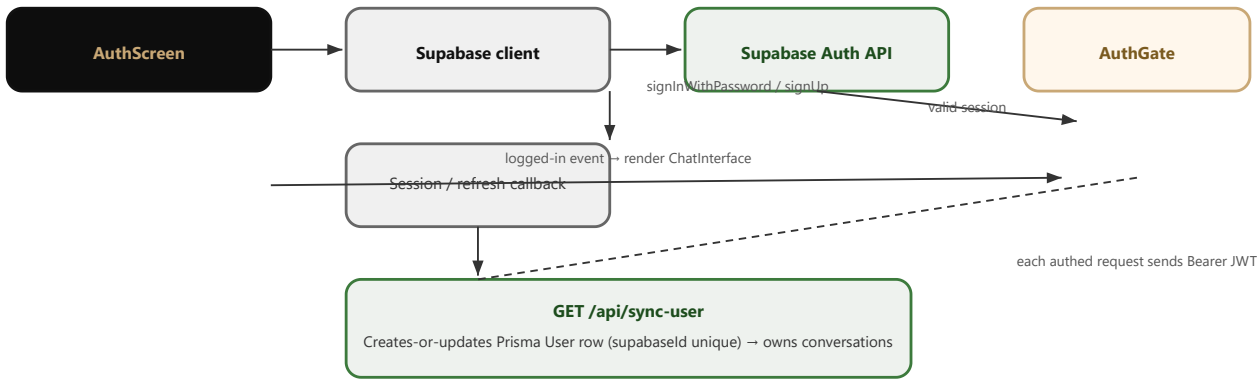
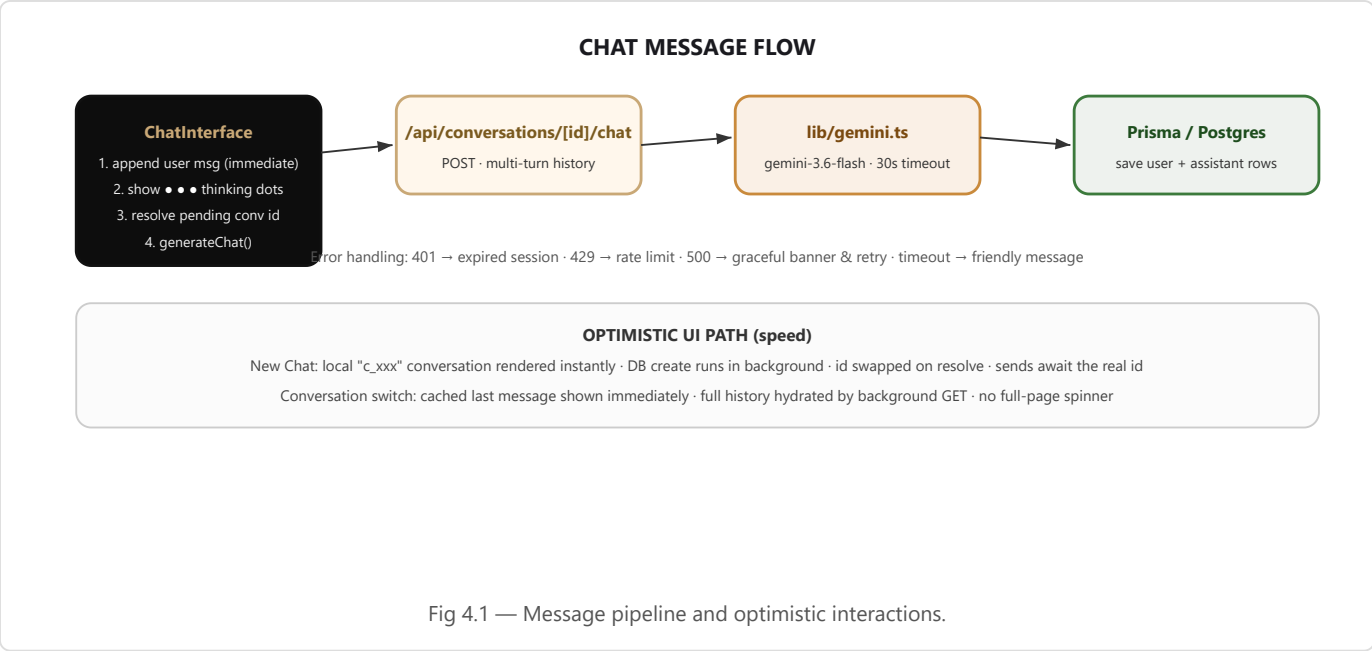
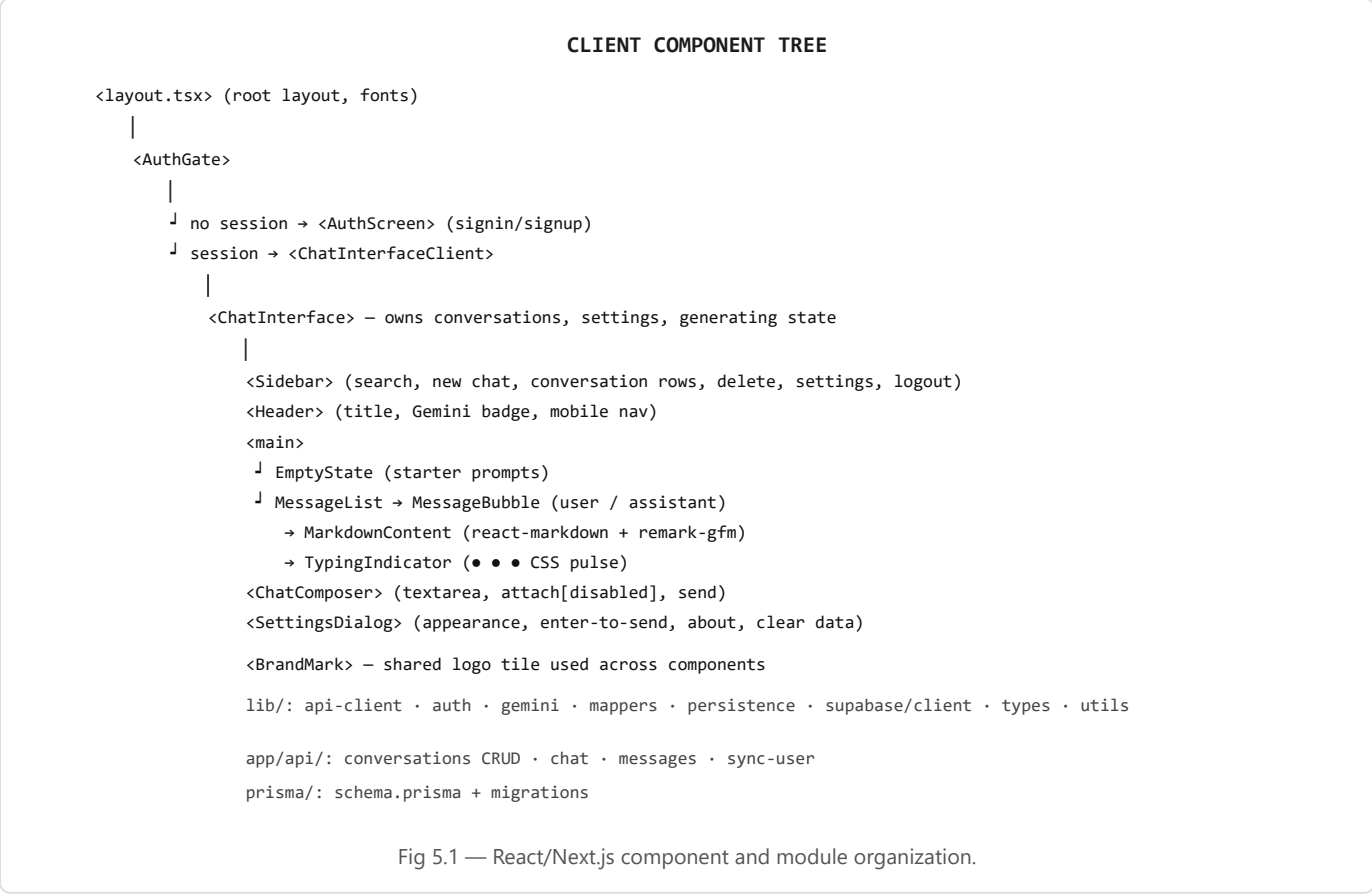


Fig 3.1 — Sign-in/sign-up flows, session propagation, and user provisioning.

4. Chat Message Flow (Send → Gemini → Persist)



5. React Component Hierarchy

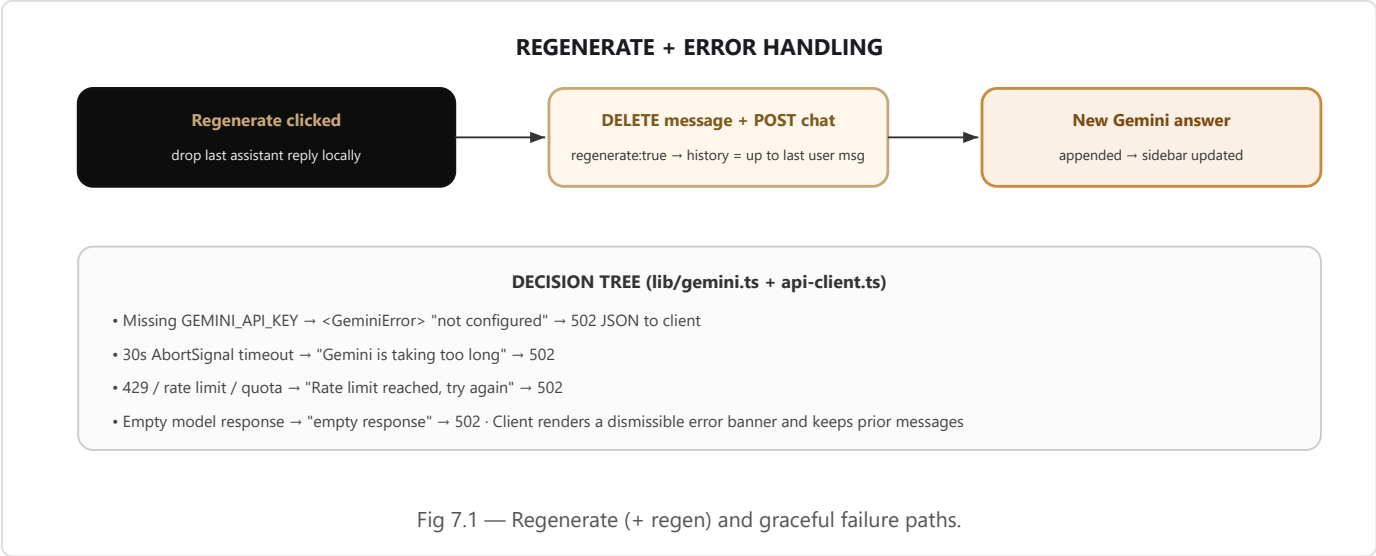


6. API Route Map

Method	Endpoint	What it does
GET	/api/conversations	List the signed-in user's conversations, newest first (with latest message preview).
POST	/api/conversations	Create a new empty conversation ("New chat"); returns the persisted row.

GET	/api/conversations/[id]	Return one conversation with its full ordered message history.
PATCH	/api/conversations/[id]	Update a conversation (title).
DELETE	/api/conversations/[id]	Delete a conversation and (cascade) its messages.
POST	/api/conversations/[id]/messages	Append a single user/assistant message.
DELETE	/api/conversations/[id]/messages/[messageId]	Remove a message (used by Regenerate).
POST	/api/conversations/[id]/chat	Save user turn, call Gemini with full history, persist assistant reply; returns both rows + title.
GET	/api/sync-user	Provision the Prisma User row from the Supabase identity.

7. Regenerate & Error-Handling Flow



8. Interaction Flow Summary

Interaction	New behaviour	Why it is fast
New Chat	Empty conversation appears instantly; NPC created in background	Optimistic local id swapped for server id on resolve; composer auto-focused
Switch conversation	Known messages render immediately; history hydrates in background	No full-page spinner; per-conversation load dedup (Set of loaded ids)
Send message	User bubble + thinking dots appear instantly	UI updates before the fetch; does not wait for server
Thinking indicator	Three bronze dots, CSS opacity pulse, staggered 0/160/320 ms	GPU-composited; disabled under prefers-reduced-motion
Response render	Markdown: headings, lists, bold/italic, code blocks, links	react-markdown + remark-gfm, no dangerouslySetInnerHTML
Persistence	All turns saved to Postgres per user, multi-device	Server-owned ownership checks; cascade deletes